

Real Estate NLP Platform

Written project report

NLP Internship | September 2026

This report documents a natural-language real-estate search platform that turns free-form buyer requests and listing remarks into structured filters, searchable documents, extracted entities, summaries, and compliance-review signals. The main conclusion is that the project has a usable service boundary and strong structured-field extraction on the available evaluation samples, while retrieval quality and test reproducibility remain the main engineering priorities.

At a glance	Evidence
API surface	12 application routes across system, NLP, search, and operations
Data scale	204,912 raw listings; 100-listing processed evaluation sample
Retrieval	Keyword precision@1 0.580; keyword MRR 0.661 on 50 queries
Extraction	Bedroom and bathroom entity accuracy 1.000; price accuracy 0.880 on 100 listings
Release risk	70%+ line-coverage gate is pending test-fixture and dependency repair

1 Introduction

Real-estate search interfaces usually expose structured fields while users describe needs in natural language. This project addresses that gap with a small service-oriented platform. A query such as “3 bed 2 bath under 700k in Irvine with a pool” becomes structured constraints that can filter a listing corpus and produce an inspectable response. Listing text also flows through cleaning, entity extraction, taxonomy matching, summarization, and fair-housing compliance checks.

The work is deliberately pragmatic. The production API uses a deterministic token-overlap ranking path and can operate with regex fallbacks when optional models or project data are not available. A separate semantic-search module and evaluation artifacts support future ranking improvements, but the current production surface favors predictable behavior, simple deployment, and observable failure modes.

2 System architecture

2.1 Application flow

The Streamlit interface collects a natural-language query and calls FastAPI over JSON. FastAPI middleware assigns request IDs, applies a per-IP sliding-window limit, logs structured request events, and returns rate-limit headers. The endpoint delegates to NLPServices, a lazy facade that loads the parser, extractor, and compliance checker on demand. Search results are cached in a bounded TTL cache, and corpus replacement or database reload invalidates stale entries.

The database loader maps the MLS-style `rets_property` table to a normalized document shape with listing ID, remarks, address, city, bedrooms, bathrooms, and price. The application can also accept documents directly through `PUT /documents`, which makes local testing independent of MySQL. Readiness reports database state separately from liveness.

Component	Role	Implementation
FastAPI service	REST boundary, validation, middleware, OpenAPI	scripts/main.py
Query parser	Price, bedroom, bathroom, city, amenity, and exclusion filters	scripts/query_parser.py
Entity extractor	Numeric signals and taxonomy entities	scripts/entity_extractor.py
Search	Token-overlap ranking plus structured filter matching	NLPServices.search
Streamlit UI	Human-facing query form and listing display	scripts/frontend/search.py
MySQL loader	Atomic corpus replacement from rets_property	NLPServices.load_documents_from_database
Compliance checker	Fair-housing rule findings and review guidance	scripts/compliance_checker.py

3 API and data contract

The API uses Pydantic request models with length and range validation. A search document has a required string id, required text, and extensible metadata object. Search requests may use the server corpus or provide a request-local document list. The parser can return parameterized SQL for inspection, which keeps values separate from the SQL text and provides a testable safety boundary.

The main routes are POST /search, POST /parse-query, POST /extract-entities, POST /summarize, and POST /check-compliance. System routes include /health and /ready. Operations routes expose cache statistics and clearing, while document routes support replacement and MySQL reload. Full payload examples and configuration variables appear in docs/API.md and docs/data-schema.md.

4 Evaluation design

The repository includes evaluation files rather than a single aggregate score. Retrieval evaluates ranking and reciprocal rank on 50 queries. Structured extraction checks bedrooms, bathrooms, and square footage on 40 examples. Entity evaluation checks numeric fields on 100 listings. Taxonomy evaluation compares gold features with matched and missed predictions across 100 listings.

The evaluation is best interpreted as a baseline. The search comparison shows a meaningful gap between semantic and keyword ranking in the stored results, while the API's production path is deterministic token overlap. The metrics therefore describe the committed evaluation pipeline, not a promise that every deployed configuration uses the same ranker.

5 Results

Metric	Result	Sample / interpretation
Keyword precision@1	0.580	50 queries; relevant item is first in 29 queries
Keyword MRR	0.661	50 queries; rewards relevant items near the top
Semantic precision@1	0.260	50 queries; current stored semantic

		baseline
Bedroom accuracy	0.750	40 structured checks
Bathroom accuracy	0.675	40 structured checks
Square-footage accuracy	0.825	40 structured checks
Entity bedroom accuracy	1.000	100 listings
Entity bathroom accuracy	1.000	100 listings
Entity price accuracy	0.880	100 listings
Taxonomy recall	0.770	191 of 248 gold features matched

The strongest result is numeric entity extraction for bedrooms and bathrooms, where every one of the 100 evaluated rows is marked correct. Price extraction is also strong but not perfect. Taxonomy recall of 0.770 indicates that canonicalization and feature coverage still miss a material share of gold features. Retrieval is the larger product risk: a keyword MRR of 0.661 is useful for a prototype, but users will notice ranking errors when several homes share common terms.

6 Reliability, safety, and deployment

The Dockerfile uses python:3.12-slim, creates a non-root api user with UID 10 001, installs requirements before copying source for layer caching, exposes port 8000, and defines a health check against /health. docker-compose.yml provisions MySQL 8.0 with a named volume and read-only raw-data initialization mount. The API supports environment variables for database connectivity, cache limits, rate limits, CORS origins, and corpus paths.

Operational controls are intentionally small but concrete. The cache is thread-safe, bounded, and TTL-based. The rate limiter returns HTTP 429 with Retry-After. Request logs include method, path, status, duration, request ID, and client IP. Production deployment would still need authentication, secret management, TLS, centralized logs, vulnerability scanning, and a formal database migration path.

The compliance checker should be treated as a review aid. It identifies rule matches and severity but does not replace legal or policy review. The roadmap therefore includes domain-expert review, versioned rules, and an audit trail.

7 Test health and coverage

The repository contains a substantial pytest suite spanning cleaning, parsing, extraction, intent classification, evaluation utilities, compliance, and API-adjacent behavior. The focused run on 11 September 2026 reported 264 passing tests, 8 failures, and 1 fixture error. Full collection also depends on a locally available sentence-transformers model and MySQL assumptions.

The failures are documented rather than hidden. Three query-parser tests expect canonical aliases that the current parser returns in surface form. One test expects 1.2m price normalization through the signal extractor. One fixture expects quartz countertop. Three numeric evaluation tests expect legacy columns that are not present in the current CSV. One answerability test references an undefined checker fixture. The repository also lacks coverage.py and pytest-cov, so no defensible line-coverage percentage is claimed. The release gate should be implemented after repairing these issues and mocking external boundaries.

8 Conclusion and roadmap

The project demonstrates a coherent end-to-end path from natural-language real-estate intent to structured, searchable, and reviewable output. The API and deployment artifacts are sufficiently concrete for local demonstration. The evidence supports shipping the platform as a prototype with explicit limitations: ranking needs improvement, taxonomy recall needs attention, and automated coverage is not yet release-ready.

Near-term work should repair fixture drift, add route contract tests, and make the semantic model reproducible offline. Medium-term work should evaluate a hybrid BM25 plus embedding ranker, version data schemas, and add latency and cache observability. Longer-term work should add human feedback for taxonomy expansion, calibrate confidence, and establish expert-reviewed compliance governance.

Appendix A Demo path

The recording-ready demo uses /docs: health and readiness, parse a constrained home-search query, search the local corpus, repeat to show a cache hit, extract listing entities, and run a compliance check. The full narration is in docs/demo-video-script.md. A binary video is not included because this workspace has no recording or publishing target.

Step	Endpoint or screen	Expected evidence
1	GET /health and GET /ready	Liveness, readiness, document count, and database status
2	POST /parse-query	Structured price, bedroom, bathroom, city, and amenity filters
3	POST /search	Ranked listings, scores, metadata, and X-Cache: MISS
4	Repeat POST /search	Same response path with X-Cache: HIT
5	POST /extract-entities	Normalized numeric fields and canonical taxonomy entities
6	POST /check-compliance	Review signal with category and severity